CHAPTER

7

ARRAYS IN PYTHON

Suppose there is a group of students whose marks are to be listed. The first student's marks are stored in a variable m1, the second student's marks are stored in m2, the third student's marks in m3, and so on and the 100th student's marks are stored in m100. It means, we are supposed to create 100 variables to store 100 students' marks. Thus, we are supposed to write 100 statements like this:

```
1=65
m2=61
m3=70
:
m100=67
```

Now, if we want to display these marks, we need to write another 100 statements. It means, a simple program will contain hundreds of statements and this becomes difficult for the programmer. On the other hand, just imagine there is only one variable that stores 100 students' marks. That variable will be very much useful to us since we have to write only 1 statement instead of writing 100 statements. For example, we can declare 'm' variable as an array and store all the marks there. This reduces the program size considerably and the programmer's task will become easy. Let's now learn more about array.

Array

So, what is an array? An array is an object that stores a group of elements (or values) of same datatype. The main advantage of any array is to store and process a group of elements easily. There are two points we should remember in case of arrays in Python.

- Arrays can store only one type of data. It means, we can store only integer type elements or only float type elements into an array. But we cannot store one integer, one float and one character type element into the same array.

- Arrays can increase or decrease their size dynamically. It means, we need not declare the size of the array. When the elements are added, it will increase its size and when the elements are removed, it will automatically decrease its size in memory.

In Python, there is a standard module by the name 'array' that helps us to create and use arrays. To get help on 'array' module, we can type help() at Python prompt to get the help prompt as:

```
>>>help()
help>
```

Then we should type simply 'array' at the help prompt to see all information about the array module as shown in Figure 7.1:

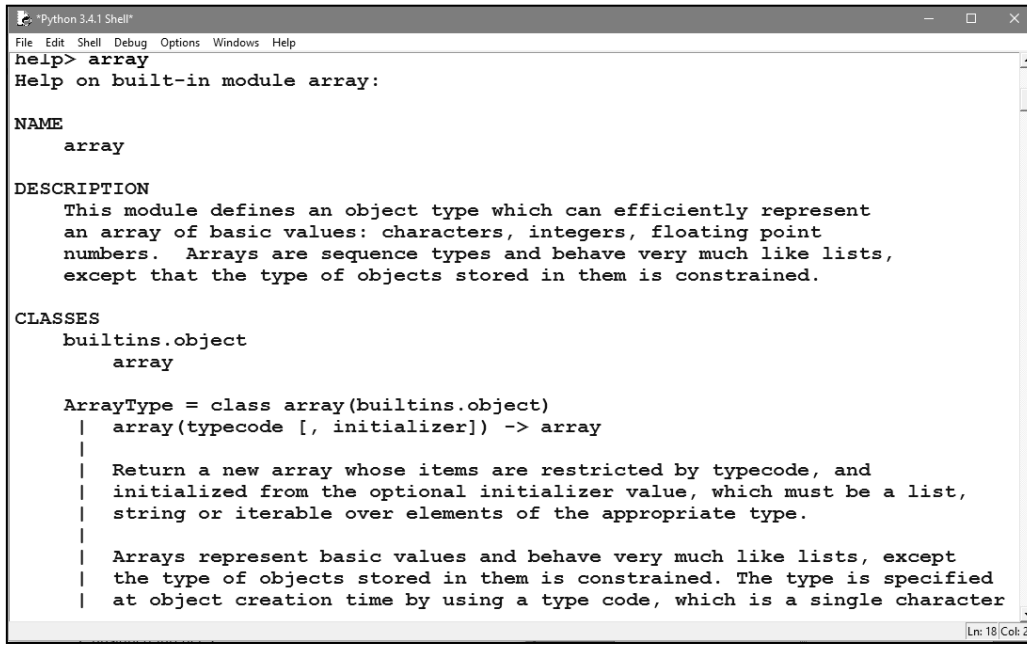


Figure 7.1: Getting Help on Array Module

Advantages of Arrays

The following are some advantages of arrays:

- Arrays are similar to lists. The main difference is that arrays can store only one type of elements; whereas, lists can store different types of elements. When dealing with a huge number of elements, arrays use less memory than lists and they offer faster execution than lists.
- The size of the array is not fixed in Python. Hence, we need not specify how many elements we are going to store into an array in the beginning.

- Arrays can grow or shrink in memory dynamically (during runtime).
- Arrays are useful to handle a collection of elements like a group of numbers or characters.
- Methods that are useful to process the elements of any array are available in 'array' module.

Creating an Array

We have already discussed that arrays can hold data of same type. The type should be specified by using a type code at the time of creating the array object as:

```
arrayname = array(type code, [elements])
```

The type code 'i', represents integer type array where we can store integer numbers. If the type code is 'f' then it represents float type array where we can store numbers with decimal point. The important type codes are given in Table 7.1:

Table 7.1: The Type Codes to Create Arrays

Typecode	C Type	Minimum size in bytes
'b'	signed integer	1
'B'	unsigned integer	1
'i'	signed integer	2
'I'	unsigned integer	2
'l'	signed integer	4
'L'	unsigned integer	4
'f'	floating point	4
'd'	double precision floating point	8
'u'	unicode character	2

We will take an example to understand how to create an integer type array. We should first write the module name 'array' and then the type code we can use is 'i' for integer type array. After that the elements should be written inside the square braces [ ] as,

```
a = array('i', [4, 6, 2, 9])
```

This is creating an array whose name is 'a' with integer type elements 4, 6, 2 and 9.

Similarly, to create a float type array, we can write:

```
arr = array('d', [1.5, -2.2, 3, 5.75])
```

This will create an array by the name 'arr' with float type elements 1.5, -2.2, 3.0 and 5.75. The type code is 'd' which represents double type elements each taking 8 bytes memory.

Of course, in the previous statements we are using 'array' module to create arrays. To use this module, we have to first import 'array' module into our program.

Importing the Array Module

There are three ways to import the array module into our program. The first way is to import the entire array module using import statement as,

```
import array
```

When we import the array module, we are able to get the 'array' class of that module that helps us to create an array. See the following example:

```
a=array.array('i',[4,6,2,9])
```

Here, the first 'array' represents the module name and the next 'array' represents the class name for which the object is created. We should understand that we are creating our array as an object of array class.

The second way of importing the array module is to give it an alias name, as:

```
import array as ar
```

Here, the array is imported with an alternate name 'ar'. Hence we can refer to the array class of 'ar' module as:

```
a=ar.array('i', [4,6,2,9])
```

The third way of importing the array module is to write:

```
from array import *
```

Observe the '*' symbol that represents 'all'. The meaning of this statement is this: import all (classes, objects, variables etc) from the array module into our program. That means we are specifically importing the 'array' class (because of * symbol) of 'array' module. So, there is no need to mention the module name before our array name while creating it. We can create the array as:

```
a=array('i',[4,6,2,9])
```

Here, the name 'array' represents the class name that is available from the 'array' module. Let's write a Python program to create an integer type array with some integer elements. Program 1 shows how to create an integer type array.

Program

Program 1: A Python program to create an integer type array.

```
# creating an array
import array
a = array.array('i', [5, 6, -7, 8])

print('The array elements are: ')
for element in a:
    print(element)
```

Output:

```
C:\>python arr.py
5
6
-7
8
```

Observe the for loop in the previous program. The variable 'element' assumes each element value from the array 'a' and hence print(element) will display all the elements of the array. The same program can be rewritten using a different type of import statement as shown in Program 2.

Program

Program 2: A Python program to create an integer type array.

```
# creating an array - v 2.0
from array import *
a = array('i', [5, 6, -7, 8])

print('The array elements are: ')
for element in a:
    print(element)
```

Output:

```
C:\>python arr.py
5
6
-7
8
```

Let's write another program to see how to create an array with a group of characters and display the elements of the array. This is shown in Program 3.

Program

Program 3: A Python program to create an array with a group of characters.

```
# creating an array with characters
from array import *
arr = array('u', ['a','b','c','d','e'])

print('The array elements are: ')
for ch in arr:
    print(ch)
```

Output:

```
C:\>python arr.py
a
b
c
d
e
```

It is possible to create an array from another array. For example, there is an array 'arr1' as:

```
arr1 = array('d', [1.5, 2.5, -3.5, 4])
```

This is a float type array with 4 elements. We want to create another array with the name 'arr2' from arr1. It means, 'arr2' should have same type code and same elements as that of 'arr1'. For this purpose, we can write:

```
arr2 = array(arr1.typecode, (a for a in arr1))
```

Here, 'arr1.typecode' gives the type code character of the array 'arr1'. This type code is used for 'arr2' array also. Observe the expression:

```
a for a in arr1
```

The first 'a' represents that its value is stored as elements in the array 'arr2'. This 'a' value is got from 'a' (each element) in arr1. So, all the elements in arr1 will be stored into arr2. Suppose, we write the expression,

```
arr2 = array(arr1.typecode, (a*3 for a in arr1))
```

In this case, the value of 'a' obtained from arr1, is multiplied by 3 and then stored as element into arr2. So, arr2 will get the elements: 4.5, 7.5, -10.5, and 12.0. This can be verified from Program 4.

Program

Program 4: A Python program to create one array from another array.

```
# creating one array from another array
from array import *
arr1 = array('d', [1.5, 2.5, -3.5, 4])

# use same type code and multiply each element of arr1 with 3
arr2 = array(arr1.typecode, (a*3 for a in arr1))
print('The arr2 elements are: ')
for i in arr2:
    print(i)
```

Output:

```
C:\>python arr.py
The arr2 elements are:
4.5
7.5
-10.5
12.0
```

Indexing and Slicing on Arrays

An *index* represents the position number of an element in an array. For example, when we create the following integer type array:

```
x = array('i', [10, 20, 30, 40, 50])
```

Python interpreter allocates 5 blocks of memory, each of 2 bytes size and stores the elements 10, 20, 30, 40 and 50 in these blocks. Figure 7.2 shows the arrangement of elements the array 'x':

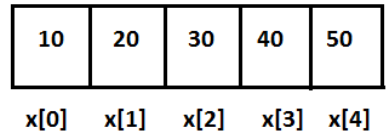


Figure 7.2: Arrangement of Elements in an Array

When we observe Figure 7.2, we can understand that the 0th element of the array is represented by x[0], the 1st element is represented by x[1] and so on. Here, 0, 1, 2, etc, are representing the position numbers of the elements. So, in general we can use i to represent the position of any element. This 'i' is called 'index' of the array. Using 'index, we can refer to any element of the array as x[i] where 'i' values will change from 0 to n-1. Here n represents the total number of elements in the array.

To find out the number of elements in an array we can use the len() function as:

```
n = len(x)
```

The len(x) function returns the number of elements in the array 'x' into 'n'. In Program 5, we are showing how to access the elements of an array using index.

Program

Program 5: A Python program to retrieve the elements of an array using array index.

```
# accessing elements of an array using index
from array import *
x = array('i', [10, 20, 30, 40, 50])

# find number of elements in the array
n = len(x)

# display array elements using indexing
for i in range(n):  # repeat from 0 to n-1
    print(x[i], end=' ')
```

Output:

```
C:\>python arr.py
10 20 30 40 50
```

It is also possible to access the elements of an array using while loop. The same program can be rewritten using while loop as Program 6.

Program

Program 6: A Python program to retrieve elements of an array using while loop.

```
# accessing elements of an array using index - v 2.0
from array import *
x = array('i', [10, 20, 30, 40, 50])
```

```
# find number of elements in the array
n = len(x)

# display array elements using indexing
i=0
while i<n:
    print(x[i], end=' ')
    i+=1
```

Output:

```
C:\>python arr.py
10 20 30 40 50
```

A *slice* represents a piece of the array. When we perform 'slicing' operations on any array, we can retrieve a piece of the array that contains a group of elements. Whereas indexing is useful to retrieve element by element from the array, slicing is useful to retrieve a range of elements. The general format of a slice is:

```
arrayname[start:stop:stride]
```

We can eliminate any one or any two in the items: 'start', 'stop' or 'stride' from the above syntax. For example,

```
arr[1:4]
```

The above slice gives elements starting from 1st to 3rd from the array 'arr'. Counting of the elements starts from 0. All the items 'start', 'stop' and 'stride' represent integer numbers either positive or negative. The item 'stride' represents step size excluding the starting element.

To understand how one can perform slicing operations on an array, we will write a Python program where we take an array 'x' and do slicing on the elements of the array 'x'. The result of slicing operations will be a new array 'y'. Please go through Program 7.

Program

Program7: A Python program that helps to know the effects of slicing operations on an array.

```
# create array y with elements from 1st to 3rd from x
y = x[1:4]
print(y)

# create array y with elements from 0th till the last element in x
y = x[0:]
print(y)

# create array y with elements from 0th to 3rd from x
y = x[:4]
print(y)

# create array y with last 4 elements from x
y = x[-4:]
print(y)

 # create y with last 4th element and with 3 [-4-(-1)= -3]elements
# towards right.
```

```
y = x[-4: -1]
print(y)

# create y with 0th to 7th elements from x.
# stride 2 means, after 0th element, retrieve every 2nd element from x
y = x[0:7:2]
print(y)
```

Output:

```
C:\>python arr.py
array('i', [20, 30, 40])
array('i', [10, 20, 30, 40, 50, 60, 70])
array('i', [10, 20, 30, 40])
array('i', [40, 50, 60, 70])
array('i', [40, 50, 60])
array('i', [10, 30, 50, 70])
```

From Program 7, we can understand that slicing can be used to create new arrays from existing arrays. For example, when we write:

```
y = x[-4:]
```

we are creating a new array 'y' with the last 4 elements of 'x' that we obtained through slicing. Slicing can also be used to update an array. For example, to replace the 1st and 2nd elements of 'x' array with the elements [5, 7] of another array, we can write:

```
x[1:3] = array('i',[5, 7])
```

The elements 5, 7 are stored in the positions 1 and 2 in the array 'x'. The remaining element of the array will not be changed.

Just like indexing, slicing can also be used to retrieve elements from an array. In Program 8, we are displaying only a range of elements from the array 'x'.

Program

Program 8: A Python program to retrieve and display only a range of elements from an array using slicing.

```
# using slicing to display elements of an array.
from array import *
x = array('i', [10, 20, 30, 40, 50, 60, 70])

# display elements from 2nd to 4th only
for i in x[2:5]:
    print(i)
```

Output:

```
C:\>python arr.py
30
40
50
```

Processing the Arrays

The arrays class of arrays module in Python offers methods to process the arrays easily. The programmers can easily perform certain operations by using these methods. We should understand that a method is similar to a function that performs a specific task. But methods are written inside a class whereas a function can be written inside or outside the class. Methods are generally called as: objectname.method(). Please see Table 7.2 for methods that can be used on arrays:

Table 7.2: Array Methods

Method	Description
a.append(x)	Adds an element x at the end of the existing array a.
a.count(x)	Returns the numbers of occurrences of x in the array a.
a.extend(x)	Appends x at the end of the array a. 'x' can be another array or an iterable object.
a.fromfile(f, n)	Reads n items (in binary format) from the file object f and appends to the end of the array a. 'f' must be a file object. Raises 'EOFError' if fewer than n items are read.
a.fromlist(lst)	Appends items from the lst to the end of the array. 'lst' can be any list or iterable object.
a.fromstring(s)	Appends items from string s to the end of the array a.
a.index(x)	Returns the position number of the first occurrence of x in the array. Raises 'ValueError' if not found.
a.insert(i, x)	Inserts x in the position i in the array.
a.pop(x)	Removes the item x from the array a and returns it.
a.pop()	Remove last item from the array a.
a.remove(x)	Removes the first occurrence of x in the array a. Raises 'ValueError' if not found.
a.reverse()	Reverses the order of elements in the array a.
a.tofile(f)	Writes all elements to the file f.
a.tolist()	Converts the array 'a' into a list.
a.tostring()	Converts the array into a string.

Apart from the methods shown in Table 7.2, we can also use the following variables of Array class as shown in Table 7.3:

Table 7.3: Variables of Array Class

Variable	Description
a.typecode	Represents the type code character used to create the array a
a.itemsize	Represents the size of items stored in the array (in bytes)

Most of these methods are used in Program 9 so that there will be better clarity regarding their results.

Program

Program 9: A Python program to understand various methods of arrays class.

```
# operations on arrays
from array import*

# create an array with int values
arr = array('i', [10,20,30,40,50])
print('Original array: ', arr)

# append 30 to the array arr
arr.append(30)
arr.append(60)
print('After appending 30 and 60: ', arr)

# insert 99 at position number 1 in arr
arr.insert(1, 99)
print('After inserting 99 in 1st position: ', arr)

# remove an element from arr
arr.remove(20)
print('After removing 20: ', arr)

# remove last element using pop() method
n = arr.pop()
print('Array after using pop(): ', arr)
print('Popped element: ', n)

# finding position of element using index() method
n = arr.index(30)
print('First occurrence of element 30 is at: ', n)

# convert an array into a list using tolist() method
lst = arr.tolist()
print('List: ', lst)
print('Array: ', arr)
```

Output:

```
C:\>python arr.py
Original array:  array('i', [10, 20, 30, 40, 50])
After appending 30 and 60:  array('i', [10, 20, 30, 40, 50, 30, 60])
After inserting 99 in 1st position:  array('i', [10, 99, 20, 30, 40,
   50, 30, 60])
After removing 20:  array('i', [10, 99, 30, 40, 50, 30, 60])
Array after using pop():  array('i', [10, 99, 30, 40, 50, 30])
Popped element:  60
First occurrence of element 30 is at:  2
List:  [10, 99, 30, 40, 50, 30]
Array:  array('i', [10, 99, 30, 40, 50, 30])
```

Let's write a program to accept marks of a student and find the total marks and percentage of marks. This is done in Program 10. In this program, we first use input() function to accept marks into a string 'str' as:

```
str = input('Enter marks: ').split(' ')
```

When this statement is executed, the user should type marks separating them by a space as indicated by split(' ') method. These marks are stored into the string 'str'. Then we should convert this string into an array as:

```
marks = [int(num) for num in str]
```

Here, 'marks' is the name of the array. Observe the for loop at right hand side. It takes each number from 'str' and stores it into 'num'. The 'num' value is converted into 'int' using int(num). This integer number is then stored into 'marks' array. To represent that 'marks' is an array, we used square brackets for the expression at right side. These brackets store the numbers as a list into 'marks' array.

To find total marks, we can use another for loop as:

```
for x in marks:
    print(x)
    sum+=x
```

This for loop is doing two things. It is printing 'x' value that represents each element of the 'marks' array and then adding that 'x' value to sum. So, finally, 'sum' represents the total marks of the student.

To find percentage, we divide the 'sum' using number of element as:

```
percent =  sum/n
```

Here, 'n' represents the number of elements in the 'marks' array which can be obtained by calling the len() function as len(marks).

Program

Program 10: A Python program to storing student's marks into an array and finding total marks and percentage of marks.

```
from array import *
# accept marks as a string
str = input('Enter marks: ').split(' ')
```

```
# store the marks into 'marks' array
marks = [int(num) for num in str]

# display the marks and find total
sum=0
for x in marks:
    print(x)
    sum+=x
print('Total marks: ', sum)

# display percentage
n = len(marks)   # n = no. of elements in marks array
percent =  sum/n
print('Percentage: ', percent)
```

Output:

```
C:\>python arr.py
Enter marks: 50 56 75 67 45
50
56
75
67
45
Total marks: 293
Percentage: 58.6
```

We are going to develop a program to sort the elements of an array into ascending order. For this purpose, we are going to use bubble sort technique in Program 11. In this technique, all the 'n' elements from 0 to n are taken and the first element of the array x[j] is compared with the immediate element x[j+1]. If x[j] is bigger than x[j+1], then they are swapped (or interchanged) since in ascending order, we expect the smaller elements to be in the first place. When two elements are interchanged, the number of elements to be sorted becomes lesser by 1. When there are no more swaps found, the 'flag' will become 'False' and we can abort sorting. Suppose, we give the following elements for sorting:

```
Original array: 1, 5, 4, 3, 2
Compare 1 with other elements. 1 is smallest hence no swaps.
We get: 1, 5, 4, 3, 2
Compare 5 with other elements. Swap 5 with 4, 5 with 3, 5 with 2.
We get: 1, 4, 3, 2, 5
Compare 4 with other elements. Swap 4 with 3, 4 with 2.
We get: 1, 3, 2, 4, 5
Compare 3 with other elements. Swap 3 with 2.
We get: 1, 2, 3, 4, 5
```

Swapping means interchanging the values of two variables. If 'a' and 'b' are variables, we are supposed to store 'a' value into 'b' and vice versa. Swapping the values of 'a' and 'b' can be done using a temporary variable 't' as:

```
Store a value into t. i.e. t= a
Store b value into a. i.e. a = b
Store t value into b. i.e. b= t
```

These steps are shown in Figure 7.3 by taking 'a' value 1 and 'b' value 2 initially. After swapping is done, 'a' value will be 2 and 'b' value will be 1.

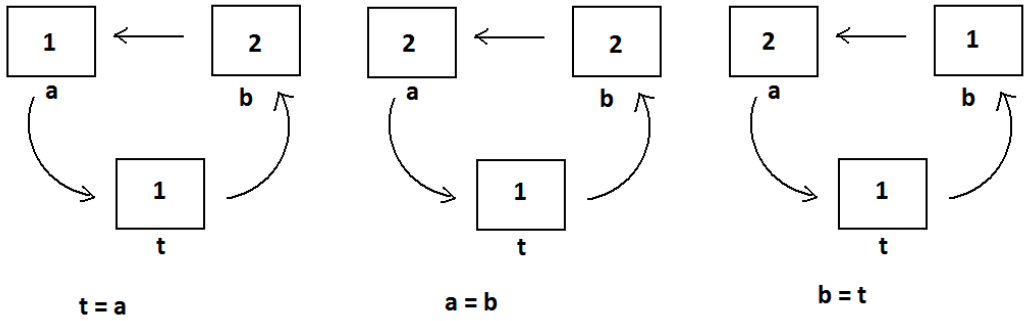


Figure 7.3: Swapping a and b Values using a Temporary Variable t

Program

Program 11: A Python program to sort the array elements using bubble sort technique.

```
# sorting an array using Bubble sort technique
from array import*
# create an empty array to store integers
x = array('i', [])

# store elements into the array x
print('How many elements? ', end='')
n = int(input()) # accept input into n

for i in range(n):  # repeat for n times
    print('Enter element: ', end='')
    x.append(int(input()))  # add the element to the array x

print('Original array: ', x)

# bubble sort
flag = False  # when swapping is done, flag becomes True
for i in range(n-1):  # i is from 0 to n-1
    for j in range(n-1-i):  # j is from 0 to one element lesser than i
        if x[j] > x[j+1]:  # if 1st element is bigger than the 2nd one
            t = x[j]  # swap j  and j+1  elements
            x[j] = x[j+1]
            x[j+1] = t
            flag = True  # swapping done, hence flag is True
    if flag==False:  # no swapping means array is in sorted order
        break  # come out of inner for loop
    else:
        flag = False  # assign initial value to flag

print('Sorted array= ', x)
```

Output:

```
C:\>python arr.py
How many elements? 5
```

```
Enter element: 1
Enter element: 5
Enter element: 4
Enter element: 3
Enter element: 2
Original array:  array('i', [1, 5, 4, 3, 2])
Sorted array:  array('i', [1, 2, 3, 4, 5])
```

Let's write another program, where we want to search for the position of an element in an array. Suppose we want to know the position of 's' in the array 'x', we have to compare 's' with each element of the array x[i] as:

```
if s == x[i]:
    print('Found at Position= ', i+1)
```

The element position is given by the index 'i'. Since array elements start from 0th position, we should add 1 to this position and display the position as i+1. Since we are comparing 's' with all elements of the array linearly, this is called 'linear search' or 'sequential search'. This logic is shown in Figure 7.4:

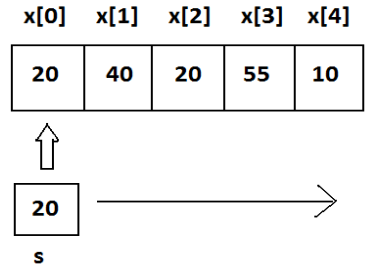


Figure 7.4: Searching for an Element in an Array using Sequential Search

Program

Program 12: A Python program to search for the position of an element in an array using sequential search.

```
# Searching an array for an element.
from array import*
# create an empty array to store integers
x = array('i', [])

# store elements into the array x
print('How many elements? ', end='')
n = int(input()) # accept input into n

for i in range(n):  # repeat for n times
    print('Enter element: ', end='')
    x.append(int(input()))  # add the element to the array x

print('Original array: ', x)

print('Enter element to search: ', end='')
s = int(input())  # accept element to be searched

# linear search or sequential search
```

```
flag=False  # this becomes True if element is found

for i in range(len(x)):  # repeat i from 0 to no. of elements
    if s == x[i]:
        print('Found at Position= ', i+1)
        flag=True
if flag==False:
    print('Not found in the array')
```

Output:

```
C:\>python arr.py
How many elements? 5
Enter element: 20
Enter element: 40
Enter element: 20
Enter element: 55
Enter element: 10
Original array: array('i', [20, 40, 20, 55, 10])
Enter element to search: 20
Found at Position= 1
Found at Position= 3
```

The same program can be rewritten using index() method provided for arrays. The index() method returns the first position of the searching element. If the element is found several times, this method returns only the first occurrence. To search for 's' in the array 'x', we can write:

```
pos = x.index(s)
    print('Found at Position= ', pos+1)
```

Here, 'pos' indicates the position number returned by index() method. If the element is not found in the array, then this method raises a runtime error called 'ValueError'. Hence, it is recommended to handle this error using try and except blocks, as:

```
try:
    statements to execute
except ValueError:
    display a message if this error raises
```

When there is a runtime error inside the try block, the control of execution jumps into except block and displays the message written there. We will discuss about runtime errors (or exceptions) in a later chapter.

Program

Program 13: A Python program to search for the position of an element in an array using index() method.

```
# searching an array for an element - v2.0
from array import*
# create an empty array to store integers
x = array('i', [])

# store elements into the array x
print('How many elements? ', end='')
n = int(input()) # accept input into n

for i in range(n):  # repeat for n times
```

```
        print('Enter element: ', end='')
        x.append(int(input()))  # add the element to the array x

    print('Original array: ', x)

    print('Enter element to search: ', end='')
    s = int(input())  # accept element to be searched

    # index() method gives the location of the element in the array
    try:
        pos = x.index(s)
        print('Found at Position= ', pos+1)

    except ValueError:  # if element not found then ValueError will rise
        print('Not found in the array')
```

Output:

```
C:\>python arr.py
How many elements? 5
Enter element: 20
Enter element: 40
Enter element: 20
Enter element: 55
Enter element: 10
Original array: array('i', [20, 40, 20, 55, 10])
Enter element to search: 20
Found at Position= 1
```

Types of Arrays

Till now, we got good idea on arrays in Python. When talking about arrays, any programming language like C or Java offers two types of arrays. They are:

- **Single dimensional arrays:** These arrays represent only one row or one column of elements. For example, marks obtained by a student in 5 subjects can be written as 'marks' array, as:

```
marks = array('i', [50, 60, 70, 66, 72])
```

  The above array contains only one row of elements. Hence it is called single dimensional array or one dimensional array.

- **Multi-dimensional arrays:** These arrays represent more than one row and more than one column of elements. For example, marks obtained by 3 students each one in 5 subjects can be written as 'marks' array as:

```
marks = array([[50, 60, 70, 66, 72],
               [60, 62, 71, 56, 70],
               [55, 59, 80, 68, 65]])
```

  The first student's marks are written in first row. The second student's marks are in second row and the third student's marks are in third row. In each row, the marks in 5 subjects are mentioned. Thus this array contains 3 rows and 5 columns and hence it is called multi-dimensional array.

Each row of the above array can be again represented as a single dimensional array. Thus the above array contains 3 single dimensional arrays. Hence, it is called a two dimensional array. A two dimensional array is a combination of several single dimensional arrays. Similarly, a three dimensional array is a combination of several two dimensional arrays.

In Python, we can create and work with single dimensional arrays only. So far, the examples and methods discussed by us are applicable to single dimensional arrays. Python does not support multi-dimensional arrays. But that is not a bad news. We can construct multidimensional arrays using third party packages like *numpy* (numerical python). The following sections are devoted for a discussion on 'numpy' which is very important for Python programmers.

Working with Arrays using numpy

numpy is a package that contains several classes, functions, variables etc. to deal with scientific calculations in Python. numpy is useful to create and also process single and multi-dimensional arrays. In addition, numpy contains a large library of mathematical functions like linear algebra functions and Fourier transforms. To get complete help on numpy, the reader can refer to the following link at scipy.org: http://docs.scipy.org/doc/numpy/reference/

The arrays which are created using numpy are called n dimensional arrays where n can be any integer. If n=1, it represents a one dimensional array. If n=2, it is a two dimensional array. Similarly, if n=3, it is a three dimensional array. The arrays created in numpy can accept only one type of elements. We cannot store different datatypes into same array.

To work with numpy, we should first import numpy module into our Python programs as:

```
import numpy
```

This will import numpy module into our program so that we can use any of the objects from that package. But, to refer to an object we should use the format: numpy.object. See the Program 14 to know how to create an array in numpy.

Program

Program 14: A Python program to create a simple array using numpy.

```
# creating single dimensional array using numpy
import numpy
arr = numpy.array([10, 20, 30, 40, 50])  # create array
print(arr) # display array
```

Output:

```
C:\>python arr.py
[10, 20, 30, 40, 50]
```

In Program 14, we used array() function to create an array with 5 elements. Please observe that we called this function as: numpy.array(). In this way, adding the word 'numpy' before every object or function would be difficult for the programmer. Hence, there is another way to write import statement, as:

```
import numpy as np
```

Here, we are using 'np' to refer to the word 'numpy'. Hence, wherever we need to type 'numpy', we simply type 'np'. This makes typing easy for the programmer as shown in Program 15.

Program

Program 15: Another version of Program 14 to create an array.

```
# creating single dimensional array using numpy - v2.0
import numpy as np
arr = np.array([10, 20, 30, 40, 50])  # create array
print(arr) # display array
```

Can't we eliminate writing even 'np' to refer to the objects or functions in our program? Yes, that is possible by writing import statement as:

```
from numpy import *
```

This will import everything (* means all) specifically from numpy package. Now, we need not use any name to refer the objects or functions in our program. See Program 16 which is another variation for the previous two programs.

Program

Program 16: Another version of Program 15 to create an array.

```
# creating single dimensional array using numpy - v3.0
from numpy import *
arr = array([10, 20, 30, 40, 50])  # create array
print(arr) # display array
```

In Program 16, we are straight away calling the function as array() and there is no need of using any word before the function.

We will see how to create arrays using numpy. We already know that an array is similar to lists except that every element of an array should be of the same datatype. Creating arrays in numpy can be done in several ways. Some of the important ways are:

- Using array() function
- Using linspace() function
- Using logspace() function
- Using arange() function
- Using zeros() and ones() functions

Creating Arrays using array()

We can call array() function of numpy module to create an array. When we create an array, we can specify the datatype of the elements either as 'int' or 'float'. We can create an integer type array as:

```
arr = array([10, 20, 30, 40, 50], int)
```

We can also eliminate 'int' in the above statement since Python can assess the datatypes of elements.

To create an array with float type elements, we should specify 'float' as:

```
arr = array([1.5, 2.5, 3, 4, -5.1], float)
```

The same statement can be rewritten by eliminating 'float' as Python can judge the datatypes of the elements. In the array, if Python interpreter finds one element belonging to 'float type', then it will convert all the other elements also into float type by adding a decimal point after the element as:

```
arr = array([10, 20, 30.1, 40])
```

If we display this array using print() function, we can see the array as:

```
[10., 20., 30.1, 40.]
```

To create an array with character type elements, we need not specify the datatype. We can simply write:

```
arr = array(['a', 'b', 'c', 'd'])
```

Program

Program 17: A Python program to create a character type array with a group of characters.

```
# creating an array with characters
from numpy import *
arr = array(['a','b','c','d'])  # create array
print(arr) # display array
```

Output:

```
C:\>python arr.py
['a' 'b' 'c' 'd']
```

To create a string type array where can store a group of strings, we should use additional attribute 'dtype = str' in the array() function as:

```
arr = array(['Delhi', 'Hyderabad', 'Mumbai', 'Ahmedabad'], dtype=str)
```

Alternately, we can omit the 'dtype=str' in the above statement and write it as:

```
arr = array(['Delhi', 'Hyderabad', 'Mumbai', 'Ahmedabad'])
```

Program

Program 18: A Python program to create a string type array using numpy.

```
# creating an array with characters
from numpy import *
# create array
arr = array(['Delhi', 'Hyderabad', 'Mumbai', 'Ahmedabad'], dtype=str)
print(arr) # display array
```

It is possible to create an array from other arrays, as shown in Program 19.

Program

Program 19: A Python program to create an array from another array.

```
# creating an array from another array
from numpy import *
a = array([1,2,3,4,5])  # original array
b = array(a)  # create b from a using array() function
c = a  # create c by assigning a to c

# display the arrays
print("a = ", a)
print("b = ", b)
print("c = ", c)
```

Output:

```
C:\>python arr.py
a = [1 2 3 4 5]
b = [1 2 3 4 5]
c = [1 2 3 4 5]
```

Creating Arrays using linspace

The linspace() function is used to create an array with evenly spaced points between a starting point and ending point. The form of the linspace() function is:

```
linspace(start, stop, n)
```

'start' represents the starting element and 'stop' represents the ending element. 'n' is an integer that represents the number of parts the elements should be divided. If 'n' is omitted, then it is taken as 50. Let's take one example to understand this.

```
a = linspace(0, 10, 5)
```

In the above statement, we are creating an array 'a' with starting element 0 and ending element 10. This range is divided into 5 equal parts and hence the points will be 0, 2.5, 5, 7.5 and 10. These elements are stored into 'a'. Please remember the staring and elements 0 and 10 are included. Program 20 shows how to create an array with 5 equal points using the linspace() function.

Program

Program 20: A Python program to creating an array with 5 equal points using linspace().

```
# creating an array using linspace()
from numpy import *

# divide 0 to 10 into 5 parts and take those points in the array
a = linspace(0, 10, 5)
print('a = ', a)
```

Output:

```
C:\>python arr.py
a = [ 0.   2.5    5.    7.5    10. ]
```

Creating Arrays using logspace

The logspace() function is similar to linspace(). The linspace() function produces the evenly spaced points. Similarly, logspace() produces evenly spaced points on a logarithmically spaced scale. The logspace() function is used in the following format:

```
logspace(start, stop, n)
```

The logspace() function starts at a value which is 10 to the power of 'start' and ends at a value which is 10 to the power of 'stop' . If 'n' is not specified, then its value is taken as 50. For example, if we write:

```
a = logspace(1, 4, 5)
```

This function represents values starting from 10^1 to 10^4. These values are divided into 5 equal points and those points are stored into the array 'a'. This can be seen in sample Program 21.

Program

Program 21: A Python program to create an array using logspace().

```
# creating an array using logspace()
from numpy import *

# divide the range: 10 power 1 to 10 power 4 into 5 equal parts
# and take those points in the array
a = logspace(1, 4, 5)

# find no. of elements in a
n = len(a)

# repeat from 0 to n-1 times
for i in range(n):
    print('%.1f' % a[i], end=' ') # display 1 digit after decimal point
```

Output:

```
C:\>python arr.py
10.0  56.2  316.2  1778.3  10000.0
```

Creating Arrays using arange() Function

The arange() function in numpy is same as range() function in Python. The arange() function is used in the following format:

```
arange(start, stop, stepsize)
```

This creates an array with a group of elements from 'start' to one element prior to 'stop' in steps of 'stepsize'. If the 'stepsize' is omitted, then it is taken as 1. If the 'start' is omitted, then it is taken as 0. For example,

```
arange(10)
```

will produce an array with elements 0 to 9.

```
arange(5, 10)
```

will produce an array with elements from 5 to 9.

```
arange(1, 10, 3)
```

will create an array with the elements starting from 1 to 9. So, the first element will be 1. Since the stepsize is 3, we should 3 to get the subsequent elements. Thus, the second element can be obtained as 1+3 = 4 and the third element can be obtained as 4+3 = 7 and so on. Hence, the array will contain the following elements: [1 4 7]

```
arange(10, 1, -1)
```

Since the stepsize is -1, it represents the elements in descending order from 10 to 2, as: [10 9 8 7 6 5 4 3 2].

The following example creates a float type array with stepsize 1.5:

```
arange(0, 10, 1.5)
```

In this case, the array elements will be: [0. 1.5 3. 4.5 6. 7.5 9.]. We will now write a Python program to create an array with even numbers up to 10. In this program, we will use the arange() function as:

```
a = arange(2, 11, 2)
```

the starting even number is 2 and everytime we are adding 2 (stepsize) to get the next even number. This will continue till 10 (one element prior to 11).

Program

Program 22: A Python program to create an array with even number up to 10.

```
# creating an array with even numbers up to 10
from numpy import *

# create an array using arange() function
a = arange(2, 11, 2)
print(a)
```

Output:

```
C:\>python arr.py
[  2  4  6  8  10]
```

Creating Arrays using zeros() and ones() Functions

We can use the zeros() function to create an array with all zeros. The ones() function is useful to create an array with all 1s. They are written in the following format:

```
zeros(n, datatype)
ones(n, datatype)
```

where 'n' represents the number of elements. we can eliminate the 'datatype' argument. If we do not specify the 'datatype', then the default datatype used by numpy is 'float'. See the examples:

```
zeros(5)
```

This will create an array with 5 elements all are zeros, as: [0. 0. 0. 0. 0.]. If we want this array in integer format, we can use 'int' as datatype, as:

```
zeros(5, int)
```

this will create an array as: [0 0 0 0 0].

If we use ones() function, it will create an array with all elements 1. For example,

```
ones(5, float)
```

will create an array with 5 integer elements all are 1s as: [1. 1. 1. 1. 1.]. Program 23 shows how to create arrays using zeros() and ones().

Program

Program 23: A Python program to create arrays using zeros() and ones().

```
# creating arrays using zeros() and ones()
from numpy import *

a = zeros(5, int)
print(a)

b = ones(5)  # default datatype is float
print(b)
```

Output:

```
C:\>python arr.py
[0  0  0  0  0]
[1.  1.  1.  1.  1.]
```

Mathematical Operations on Arrays

It is possible to perform various mathematical operations like addition, subtraction, division, etc. on the elements of an array. Also, the functions of 'math' module can be applied to the elements of the array. For example, to add the value 5 to every element of an array, we can write:

```
arr = array([10, 20, 30.5, -40])   # create an array
arr = arr+5  # add 5 to the array
```